

Introduction to Computer Architecture

Sheet #3

Questions:

1. Explain why, in MARIE, the MAR is only 12 bits wide while the AC is 16 bits wide

Ans: In the MARIE architecture:

- **MAR (Memory Address Register)** is **12 bits** wide because it is used to hold memory addresses. MARIE's memory is $2^{12} = 4096$ (4K) words, so 12 bits are sufficient to uniquely identify every memory location.
- **AC (Accumulator)** is **16 bits** wide because it is a data register. MARIE is a 16-bit word-addressable machine, meaning each data word and each instruction consists of 16 bits.

2. What is the difference between RTL and RTN?

Ans:

RTN (Register Transfer Notation): The conceptual notation used to describe the transfer of data between registers (e.g., **MBR $\rightarrow$ M[MAR]**).

RTL (Register Transfer Language): The formal language and symbolic notation used to specify the micro-operations that occur within the CPU during instruction execution.

3. How does the CPU clock affect the RTL?

Ans:

The CPU clock provides the synchronization for the system. Each RTL micro-operation is executed during a specific clock cycle. The clock frequency determines how quickly these transfers occur, effectively setting the pace for the fetch-decode-execute cycle.

Exercises:

1. For the following program:
a) List the hexadecimal code.

Hex Address	Instruction	Hex Code
100	Load A	1108
101	Add One	3109
102	Jump S1	9106
103	Add One	3109
104	Store A	2108
105	Halt	7000

Label	Hex Address	Instruction
	100	Load A
	101	Add One
	102	Jump S1
S2,	103	Add One
	104	Store A
	105	Halt
S1,	106	Add A
	107	Jump S2
A,	108	HEX 0023
One,	109	HEX 0001

106	Add A	3108
107	Jump S2	9103
108	HEX 0023	0023
109	HEX 0001	0001

b) What are the contents of the symbol table?

A	108
One	109
S1	106
S2	103

2. Given the following MARIE program, generate the symbol table from the first pass.

```

If,    100 Load    X    /Load the first value
      101 Subt     Y    /Subtract the value of Y, store result in AC
      102 Skipcond 400  /If AC=0 (X=Y), skip the next instruction
                          /Note: This is Skipcond 01 in the book
      103 Jump     Else /Jump to Else part if AC is not equal to 0
Then,  104 Load    X    /Reload X so it can be doubled
      105 Add      X    /Double X
      106 Store     X    /Store the new value
      107 Jump     Endif /Skip over the false, or else, part to end of if
Else,  108 Load    Y    /Start the else part by loading Y
      109 Subt     X    /Subtract X from Y
      10A Store     Y    /Store Y-X in Y
Endif, 10B Halt        /Terminate program (it doesn't do much!)
X,     10C Dec       12  /Assume these values for X and Y
Y,     10D Dec       20

```

Label	Hex Address
If	100
Then	104
Else	108
Endif	10B
X	10C
Y	10D

3. Consider a program with the following requirements:

- Add the contents of memory location 940H to the contents of memory location 941H.
- Store the result (held in the accumulator register) back into memory location 941H.

Assume:

- The Program Counter (PC) holds the address 300H.
- Memory location 940H contains the initial value 0003H.
- Memory location 941H contains the initial value 0002H.

Ans:

a) Write the MARIE assembly program to perform the operations.

/ Program starts at address 300H

LOAD 940 / Load the value 0003H from address 940 into AC

ADD 941 / Add the value 0002H from address 941 to AC

STORE 941 / Store the result (0005H) back into address 941

HALT / Terminate the program

b) Provide your MARIE program's machine code (hexadecimal representation).

Hex Address	Instruction	opcode	Address	Machine Code (Hex)
300	LOAD 940	1	940	1940
301	ADD 941	3	941	3941
302	STORE 941	2	941	2941
303	HALT	7	000	7000

c) Complete the following fetch-decode-execute table for all the instructions.

1.LOAD 940

Step	RTN	PC	IR	MAR	MBR	AC
Initial values		300	-	-	-	-
Fetch	MAR ← PC	300	-	300	-	-
	IR ← M[MAR]	300	1940	300	-	-
	PC ← PC+1	301	1940	300	-	-
Decode	MAR ← IR[11-0]	301	1940	940	-	-
	(Decode IR[15-12])	301	1940	940	-	-
Get Operand	MBR ← M[MAR]	301	1940	940	0003	-

Execute	$AC \leftarrow MBR$	301	1940	940	0003	0003
---------	---------------------	-----	------	-----	------	------

2.ADD 941

Step	RTN	PC	IR	MAR	MBR	AC
Initial values		301	1940	940	0003	0003
Fetch	$MAR \leftarrow PC$	301	1940	301	0003	0003
	$IR \leftarrow M[MAR]$	301	3941	301	0003	0003
	$PC \leftarrow PC+1$	302	3941	301	0003	0003
Decode	$MAR \leftarrow IR[11-0]$	302	3941	941	0003	0003
	(Decode IR[15-12])	302	3941	941	0003	0003
Get Operand	$MBR \leftarrow M[MAR]$	302	3941	941	0002	0003
Execute	$AC \leftarrow AC+MBR$	302	3941	941	0002	0005

3.STORE 941

Step	RTN	PC	IR	MAR	MBR	AC
Initial values		302	3941	941	0002	0005
Fetch	$MAR \leftarrow PC$	302	3941	302	0002	0005
	$IR \leftarrow M[MAR]$	302	2941	302	0002	0005
	$PC \leftarrow PC+1$	303	2941	302	0002	0005
Decode	$MAR \leftarrow IR[11-0]$	303	2941	941	0002	0005
	(Decode IR[15-12])	303	2941	941	0002	0005
Get Operand	-	303	2941	941	0002	0005
Execute	$MBR \leftarrow AC$	303	2941	941	0005	0005
	$M[MAR] \leftarrow MBR$	303	2941	941	0005	0005